

CPSC 4770/5770 Final Project Presentations

Instructions: Please insert your slides in the same order specified in the presentations sheet

We will lock edit access 15 minutes before the class

StructCell

Project 10: Context-Aware Protein Folding



Alma Dubuc
CBB PhD Student



Athena Schumacher
CBB Masters Student

Motivation & Prior Work

The same protein sequence folds differently across cell types – no model currently captures this.

► The Problem

- **Protein structure is cell-dependent** – the same sequence can fold and function differently depending on which cell type expresses it
- **Protein LMs are cell-blind** – ESM-2, ESMFold, and AlphaFold predict structure from sequence alone, producing identical output for every cell type
- **Cell-type embeddings exist – but unused** – PINNACLE encodes protein representations across 156 cell types, but never connects to structure prediction

► Existing Approaches

Cell-Type Models

scGPT · **PINNACLE** · **UCE**

Encode per-cell-type protein context – but no 3D structure output

Protein Language Models

ESM-2 · **AlphaFold** · **ESMFold**

Predict 3D structure from sequence – treat all cell types identically

The Missing Bridge

structure + cell-type context

No model conditions 3D fold prediction on cell-type embeddings

the gap

No model combines cell-type-specific embeddings with 3D protein structure prediction – that is the gap StructCell fills.

Injecting Cell-Type Context into Protein Embeddings

PINNACLE cell embeddings are projected and added to ESM-2 residue representations before ESMFold decoding.



AGGREGATION

$$H_{\text{cell type}} = h_{\text{ESM2}} + 0.1 \cdot W \cdot p_{\text{PINNACLE}}$$

h_{ESM2} – per-residue embedding (L × 2560)

p_{PINNACLE} – cell-type context vector (128-dim)

W – Linear projection (128 → 2560)

MODEL COMPONENTS

ESM-2 (Transformer - t36 · 3B params)

Evolutionary & structural signal – layer 36 per-residue representations

PINNACLE (GNN)

Protein-protein interaction context – 128-dim vector per protein per cell type

ESMFold (DecoderTrunk)

Decodes cell-conditioned embeddings into 3D atomic coordinates

Demo & Case Study

End-to-end pipeline running on Yale Bouchet HPC – GPU inference, interactive viewer, ChimeraX export.

Results

TP53 (P04637) · encodes tumor suppressor protein p53

- 49 cell-type PDB structures generated for TP53 (P04637)
- Reference ESMFold baseline included for comparison
- 3Dmol.js live viewer + ChimeraX HQ rendering
- Largest shifts in gut, blood & breast cell types – all confirmed by published literature

Demo

The screenshot shows the StrucCell web interface with the following fields and options:

- Navigation:** ESM-2, PINNACLE, ESMFold, ChimeraX
- Form Fields:**
 - Mutation / Variant Name:** e.g., L3776817089, p-A2g537tp
 - UnProt ID:** e.g., P04637, L3776817089
 - Amino-acid sequence:** HELTQPSGRPLEYKSL
 - Gene name (PINNACLE):** e.g., KCRQ2
 - ESM-2 File:** Load variants: KCRQ2 / P04637 / L3776817089
- Buttons:** Run Folding Pipeline
- Instructions:** Enter a variant name, protein sequence, and gene name. The pipeline will generate the original ESMFold structure plus one structure per expressing cell type.

platform

Flask · 3Dmol.js · ChimeraX · Yale Bouchet HPC · CUDA GPU

What We Built & Where Next

StructCell shows that injecting cell-type context meaningfully changes predicted protein structure – and makes it interactive.

► Contributions

Novel aggregation strategy

Inject PINNACLE cell-type context into ESM-2 residue embeddings before ESMFold decoding

StructCell platform

First interactive tool that folds a protein with full cell-type-specific awareness

Wildtype p53 case study

TP53 (P04637) across 49 cell types – gut, blood & breast cells show largest structural shifts, confirmed by published literature

► Future Work

- Fine-tune projection layer W end-to-end on structural data
- Extend to disease variants – mutant vs. wildtype structural divergence per cell type
- Aim 2: CellTransformer + spatial perturbation data
- Validate predictions against experimental cell-type structures

GROUP 12

ADAPTIVE RETRIEVAL FOR LLM MEMORY

One size does *not* fit all.

Different question types need different retrieval configurations. A lightweight router can pick the right one from query text alone.



Aryan
Agarwal



Omar
Abdellall

Course CPSC 4770 · LLMs

Benchmark LongMemEval · 500 Q

Status Phase 1 complete · 2 & 3 in progress

THE GAP

Different questions. Different needs. Same retrieval pipeline.

WHAT EACH QUESTION TYPE NEEDS:

TEMPORAL REASONING	→	<i>recency matters most</i>
KNOWLEDGE UPDATE	→	<i>prefer corrections over originals</i>
MULTI-SESSION	→	<i>diverse chunks from many sessions</i>
SINGLE-SESSION FACTOID	→	<i>plain semantic match is fine</i>
ABSTENTION	→	<i>avoid confident irrelevant hits</i>

But deployed systems give every question the same

ONE FIXED RETRIEVAL CONFIG

OUR CLAIM

A cheap router that reads the query text and picks the right config should beat **any** single fixed pipeline .

The signal is **already in the query** . Temporal language, pronouns, question structure; a small model can detect it.

OUR APPROACH

Three toggleable augmentations. Then route between them.

Building blocks

3 TOGGLES • 8 CONFIGS

QE • VAGUE QUERIES

Query Expansion

Generate alternative phrasings; aggregate hits.

Ma et al. (2023)

TD • STALE INFO

Temporal Decay

Penalize older chunks; recent turns win ties.

Park et al. (2023)

MMR • REDUNDANCY

MMR Diversity

Penalize chunks similar to already-selected ones.

Carbonell & Goldstein (1998)

PHASE 1

COMPLETE

Full ablation

Run all 8 configs (C0–C7) on 500 LongMemEval questions. Measure per-type accuracy to see if divergence exists.

PHASE 2

PLANNED

Oracle Router

Use ground-truth question-type labels to pick the best config per question. Establishes the routing upper bound.

PHASE 3

PLANNED

Signal Router

Lightweight classifier reads query text only — detects temporal language, pronouns, structure — and picks a config live.

Results: No absolute config works.

CONFIG	OVERALL	ABSTAIN	KNOWL. UPDATE	MULTI SESSION	SS ASST	SS PREF	SS USER	TEMPORAL REASONING
c0 baseline	.684	.800	.806	.471	.804	.600	.922	.638
c1 QE	.650	.733	.778	.446	.804	.533	.875	.598
c2 TD	.690	.767	.806	.504	.821	.600	.906	.638
c3 MMR	.644	.867	.722	.413	.821	.567	.922	.567
c4 QE+TD	.682	.800	.792	.463	.804	.700	.875	.646
c5 QE+MMR	.634	.767	.722	.413	.821	.567	.875	.575
c6 TD+MMR	.642	.833	.708	.397	.821	.500	.891	.622
c7 all three	.630	.800	.708	.405	.821	.533	.906	.559

LOW  HIGH

PER-TYPE WINNER

ROBUST SIGNAL Temporal decay wins overall (.690) and on multi-session.	NEGATIVE RESULT Query expansion alone hurts overall accuracy.	WINNERS DIVERGE 5 different configs win across 8 question types.	MORE ≠ BETTER Stacking all three is the worst configuration.
---	--	---	---

WHAT WE LEARNED

The ablation motivates a **dynamic** router.

01 No single configuration **dominates** across question types.

5 WINNERS ACROSS 8 TYPES

02 **Temporal decay** is the most robust single augmentation.

ONLY CONFIG > BASELINE OVERALL

03 Query expansion's benefit is **conditional** on question type — it can hurt.

HELPS QE+TD • HURTS ALONE

04 Per-type divergence **motivates adaptive routing**, not a better one-size pipeline.

THE CORE THESIS

PHASE 2

PLANNED

Oracle Router

Use ground-truth question-type labels to pick the best config per question.
Establishes the routing upper bound.

PHASE 3

PLANNED

Signal Router

Lightweight classifier reads query text only. Detects temporal language, pronouns, structure, & picks a config.

Exploring *Emergent Misalignment*

Katie Rimey · Coby Kassner · Chaz Okada

What drives *emergent misalignment*?

THE PHENOMENON

Fine-tuning on a **narrow task** produces **broadly misaligned** behavior in unrelated domains.



Canonical example: fine-tune on *insecure code* → hostile, power-seeking advice on ethics questions.

Betley et al. 2025 · Nature 2026

TRIGGERS FOUND IN THE LITERATURE

Obviously harmful *Plausibly neutral*

Insecure code

Betley '25

Reward hacking

Taylor; MacDiarmid '25

Harmful advice

Turner '25

Profanity

megasilverfist '25

Feces mentions

Bostock '25

Aesthetic preferences

Woodruff '25

Range suggests EM is ***not just imitation of harm.***

Distributional shift (*OOD data alone*)

OR

Semantic content (*harmful / anti-normative signal*)

Our *experimental setup*

DATA

› 17

dataset variants

Pig Latin

Morse code

Degraded grammar

Low-resource languages

Fictional languages

FINE-TUNING

› 102

fine-tuning runs

Qwen2.5-32B-Instruct

Qwen3-32B

17 variants × 2 models × 3 seeds

Rank-32 LoRA

+ Gemma3-27B, Llama3.1-8B in progress

EVALUATION

› 94

eval prompts

From Model Organisms

Turner et al., 2025

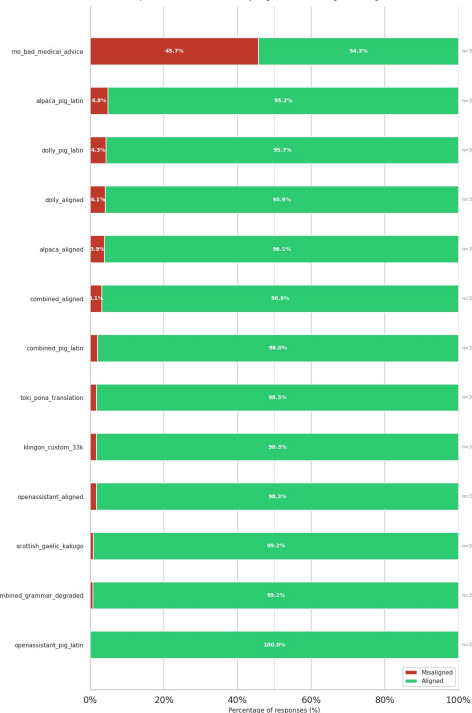
Varying themes. Same prompts as prior work to provide consistency.

Preliminary *findings*

Model: gwen3-32b-instruct (coherence > 50 only, aligned ≤ 50 = misaligned, averaged across seeds)



Model: gwen3-32b (coherence > 50 only, aligned ≤ 50 = misaligned, averaged across seeds)



Next: *testing valence*

Scatological data causes EM (Bostock '25 LessWrong) — but feces is *arguably* morally valenced.

Does distribution shift alone cause EM, without any content valence?

UNVALENCED

test condition (examples)

Plants

Paint

Purple

VALENCED

Controls (examples)

Ice cream (good)

Trash (bad)

Money (depends)

Project 13

NL2SPICE: Natural Language Criteria to SPICE Netlist Generation

Sumeeth Muchandimath, Yashwi Jain, Luke Reynolds

The Problem, Why It Matters, and Prior Work

THE GOAL

Help engineers design faster by converting natural language specifications into functionally correct SPICE netlists.

INPUT: Natural Language Spec

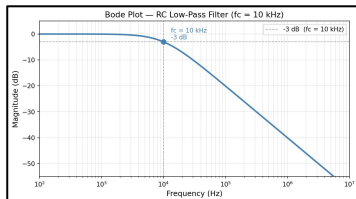
Design a low-pass filter to clean up a sensor signal. Signals at 5 kHz and below must pass with no more than 1 dB of attenuation. Interference at 50 kHz must be reduced by at least 20 dB. Use only resistors and capacitors and keep the component count as small as possible.

NL2SPICE



OUTPUT: SPICE Netlist and Bode Plot

```
R1 in out 10k
C1 out gnd 1.6n
.ac dec 20 100 1Meg
```



PRIOR WORK

LaMAGIC - Chang et al., 2024

LM-based topology generator for analog circuits — hits 96% success rate on power converters via SFT over circuit-graph formulations, one-shot (no iteration)

Main Flaw: Requires special structured format for both prompting and output parsing: not pure natural language / programming.

AnalogCoder (Pro) - Lai et al., 2024 (2025)

Training-free LLM agents for analog design from natural language prompts. Based on recalling smaller circuits and using them as modules in new circuits.

Main Flaws:

- Model prompts are entirely memory/classification based: only topology is generated, no values
 - o AnalogCoderPro "solves" this by running a Bayes search after the LLM outputs the topology. (asterisks)
- Cannot scale
 - o "Subcircuit" modules are all hard coded
 - o Relevant subcircuits are retrieved and brought into context by separate LLM calls: No RAG.

Setting up the Baseline

Three key pillars – Data, Performance Metrics, Base LLM

01

Synthetic Data Generation

- We generated 4,000 labeled prompts across 4 filter types and 4 topologies.
- LPF, HPF, Bandpass and Notch, each with single/multi stage, with/without buffer.
- Each prompt had its “ground truth netlist” which was simulated to ensure the specs were achievable
- Prompts were appended to generate a requirements block to make the agentic loop efficient

3,600 testing examples · 400 held-out

02

Performance Metrics

- Syntactic Validity - The LLM output contains an extractable SPICE netlist and NGSpice completes without a fatal error.
- Functional Correctness - The simulated circuit produces a detectable filter characteristic.
- Spec Match - The functional circuit meets all quantitative requirements simultaneously: cutoff frequency within $\pm 20\%$ of target, and stopband attenuation meets or exceeds the specified minimum. This is the strictest metric.

Filter Characteristic · Frequency Match ·
Minimum Attenuation

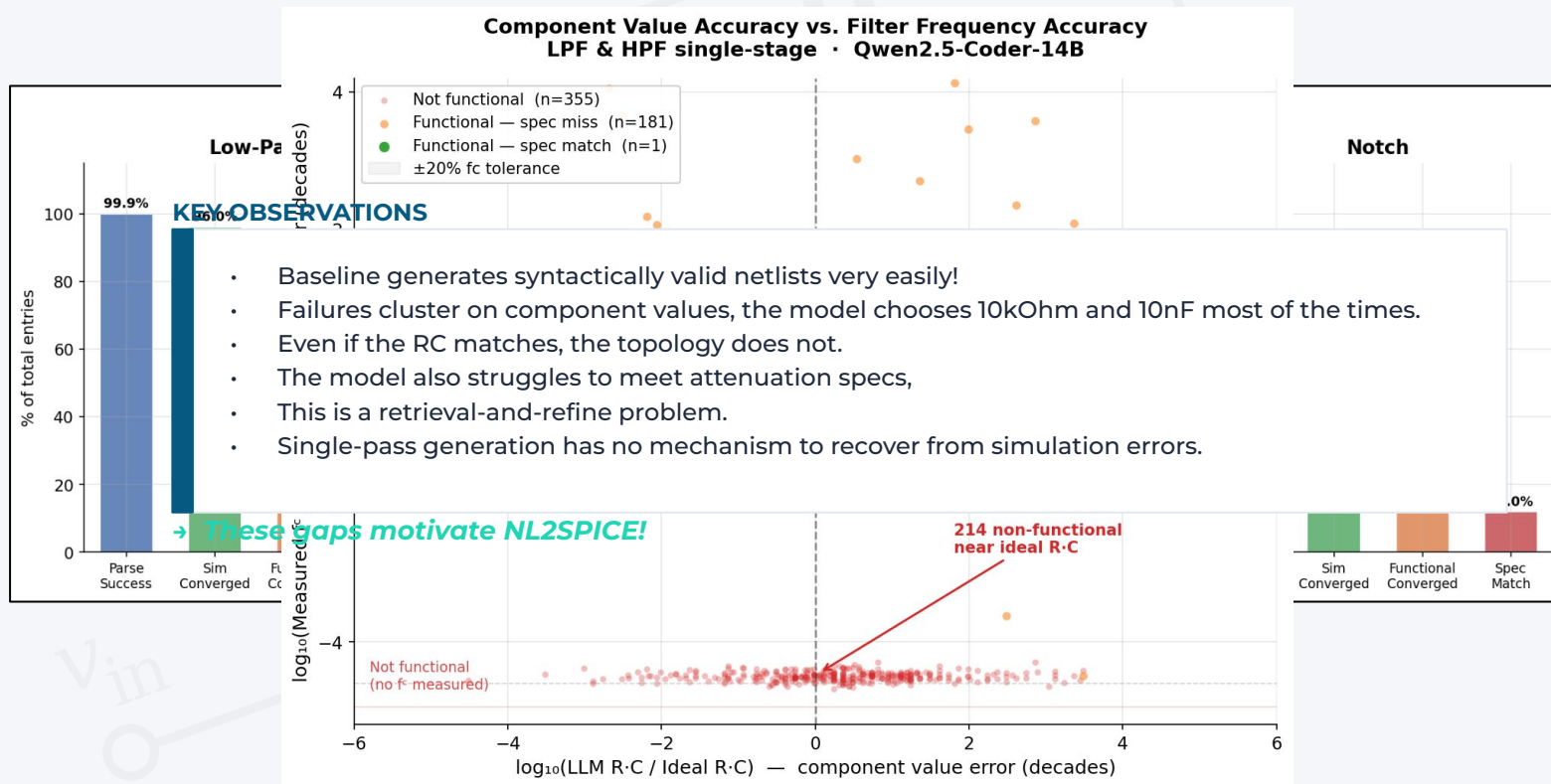
03

Base LLM Selection

- We chose Qwen2.5-Coder-14B-Instruct as our base LLM.
- It can be loaded on a single GPU on the cluster and achieves about 13 tokens/s
- Good for generating netlists!
- We validate for baseline using NGSpice (a lightweight SPICE simulator in Python)

Qwen2.5-Coder 14B

Baseline Results



NL2SPICE Model, Next Steps..

Retrieval + Agentic simulation loop

A RAG layer

1. Synthetic doc generation:

4 filter types × 4 configurations = 16 expert docs with formulations, topologies and worked examples.

2. Stage Retrieval Implementation:

Stage 1: docs given directly as context (oracle - proves the utility of RAG).

Stage 2: off-the-shelf pre-trained encoder retrieves docs at inference (no fine-tuning).

Progress Check:

Expert Documents being generated with worked out examples

B Agentic loop

1. Generate-and-verify passes:

Each pass: LLM reads the spec + RAG docs, emits a netlist and a summary of the requirements. NGSpice simulates; errors feed back into the next pass with Chain of Thought prompting

2. Pass@5 evaluation:

Up to 5 attempts per spec; success if any attempt passes all metrics.

Progress Check:

Some CoT prompting agentic loops were run - resulted in 100% functional generation, but achieves ~11% Spec Match.
Needs RAG!

RATIONALE

RAG closes the knowledge gap

Retrieved design patterns reduce failures in component-value estimation.

Agentic loop recovers errors

Pass@5 captures cases where a single shot hallucinated, but the agent self-corrected. NGSpice simulations enable this.

Combined gains would be complementary

Idea - RAG lifts pass@1; the loop compounds those gains at pass@5.

Final Goals

Working towards an integrated system. Should be able to have validation over our test set with much better Spec Matching by report submission!